Aftermath: Hackathon Project Plan

Oct 8, 2026 · @Satyabrata Das

Project overview

Aftermath is an autonomous incident-response agent: when production breaks, it reproduces the bug in a sandbox, writes and tests a fix, and opens a pull request with a post-mortem, with no human in the loop until review.

Pitch: "From crash alert to verified pull request in minutes."

Track: Coding and Agentic Engineering (NVIDIA / Nebius hackathon).

How it works, in one pass:

1. An alert (Sentry or Datadog style webhook) arrives with a stack trace.
2. The agent pulls the relevant repo files and researches the error.
3. It spins up an isolated sandbox and reproduces the failure.
4. It writes a patch, then loops: run tests, read failures, revise.
5. Only when the failing test passes and existing tests still pass does it open a PR with an AI-written post-mortem.

What makes it more than a wrapper: the patch is never submitted on the model's word. A fix must pass a test that failed before (fail-to-pass) and must not break existing tests (pass-to-pass). That verification loop is the core of the demo and of our pitch to judges.

Prize targets

Every prize requires that the project runs on Nebius Token Factory or Nebius AI Cloud and uses at least one NVIDIA open source model. We use Nemotron 3 Ultra for hard reasoning and patch writing, and Nemotron Nano or Super for fast calls like parsing and summaries. We submit under one track: Coding and Agentic Engineering.

Prize	Reward	Why we can win	What we must do
Coding and Agentic Engineering track winner	1 NVIDIA Jetson Orin Nano	The track asks for agents that write, run and test code, which is exactly Aftermath	Submit under this track; make the verified-fix loop the centre of the demo
Best Use of Tavily	$3,000	Tavily is the agent's research step, not decoration	Search for error causes and library docs, cite the sources in the PR post-mortem, show it in the video; check the Official Rules for how it is scored
Grand, 2nd, 3rd place	$20,000, $10,000, $6,000	Stretch goal against about 20,000 participants; decided by polish, demo quality and a trustworthy verification story	Same entry, executed very well
Most Valuable Feedback	$100 each, 10 winners	Feedback on Nebius and NVIDIA tools is a required part of the submission anyway	Keep notes while building; write specific, honest feedback
City Winner	$500 each, 20 winners	Only if one of us attended a Builders and Brews event	Pick the city when submitting

Not targeting: the Best Apps and Agents, Personal AI and Physical AI tracks.

Goals and success criteria

The goal is one reliable end-to-end demo on a single seeded bug, not broad coverage of every bug type.

Must have (MVP):

- A webhook endpoint that accepts a stack-trace payload and starts a run.

- A sandbox that clones a demo repo and runs its tests.
- An agent loop on Nebius that reads code, proposes a patch, and retries on test failure.
- Verification: the new test passes (fail-to-pass) and the old suite still passes (pass-to-pass).
- A real GitHub pull request with a generated post-mortem in the description.

Should have:

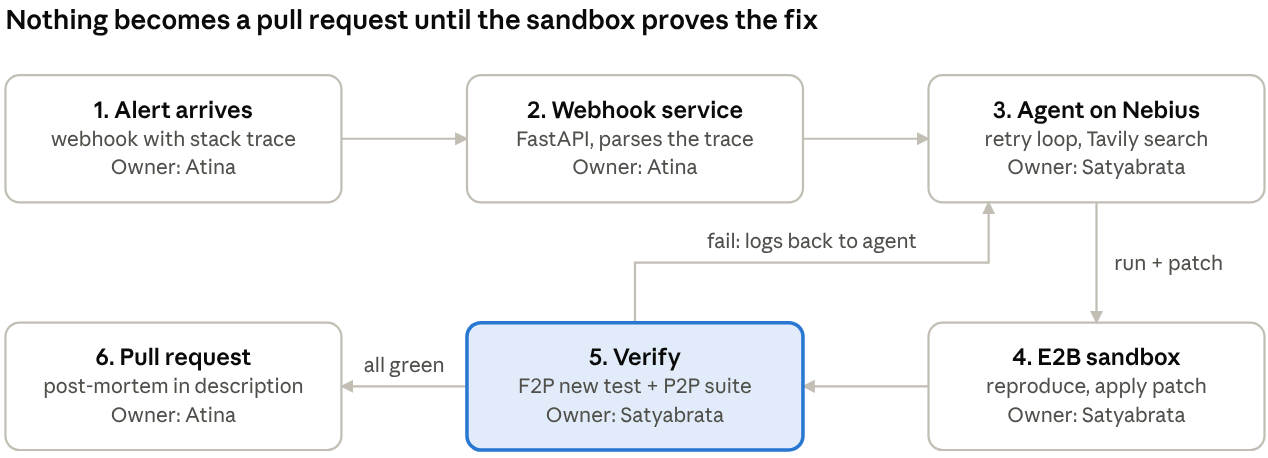
- Tavily lookups for unfamiliar errors or library docs during diagnosis, with sources cited in the post-mortem (our Best Use of Tavily entry).
- A live progress view (terminal or small web page) that streams each agent step.
- Two or three different seeded bugs (for example a null reference, an off-by-one, a bad API response handler).

Nice to have:

- An automated review pass on the patch before the PR opens.
- Run metrics: iterations, time to fix, tokens used.

We win if a judge watches a crash trigger the agent and sees a green, verified PR appear within a few minutes, and can see why the fix is trustworthy.

Architecture



architecture · 6 steps with a retry loop

The top row takes an alert to the agent; the bottom row is where the agent proves its work, and a failed test sends the logs back for another attempt.

Layer	Choice	Notes
Inference	Nebius Token Factory (OpenAI-compatible API)	Required: at least one NVIDIA open source model. Nemotron 3 Ultra for reasoning and patches, Nano or Super for fast calls; confirm names in the first model check
Web search	Tavily	Used when the agent meets an unfamiliar error or library behavior
Sandbox	E2B (Daytona as fallback)	Isolated environment; the agent never runs code on our machines
Backend	Python and FastAPI	Webhook, run queue, orchestration
Verification	pytest on the demo app	Fail-to-pass test plus the full existing suite
Source control	GitHub API	Branch, commit, pull request
Live view	Terminal output or a small web page	Streams each agent step during the demo

Setup checklist

We already have two of the keys; the rest takes about half a day. Tick items off as you finish them. Owners match the work split below.

- [x] ~~Nebius Token Factory API key (have it)~~
- [x] ~~Tavily API key (have it)~~
- [ ] **Satyabrata:** Call the Nebius models endpoint and write down which models are available and their context limits. Test Nemotron 3 Ultra for patch generation and Nemotron Nano or Super for fast calls; confirm the exact model names in the console.
- [ ] **Satyabrata:** Create the E2B account and API key (Daytona is the fallback). Run one hello-world command in a sandbox to confirm it works.
- [ ] **Satyabrata:** Create the Aftermath GitHub repo with an open source license (MIT or Apache 2.0), the separate demo target repo, and a fine-grained access token limited to the demo repo (contents and pull requests only). Invite Atina to both. Share the token

privately, never in chat or in the repo.

- [ ] **Atina:** Install Python 3.11+, then complete a short FastAPI and pytest tutorial (two to three hours).
- [ ] **Atina:** Install a tunnel (ngrok or cloudflared) so a webhook can reach her laptop while testing.
- [ ] **Both:** Python environment with the OpenAI-compatible client for Nebius and the Tavily client.
- [ ] **Both:** Shared `.env.example` listing every variable (`NEBIUS_API_KEY`, `TAVILY_API_KEY`, `E2B_API_KEY`, `GITHUB_TOKEN`, `WEBHOOK_SECRET`). Real keys stay in each person's local `.env` and are never committed.
- [ ] **Both:** Read the Official Rules together once, and check whether free Nebius credits apply to us.
- [ ] **Both:** Agree on a repo layout, a branch rule (small pull requests into `main`, review each other's), and a daily 15-minute sync.

Work split

Satyabrata takes the parts where a mistake is costly or the behavior is unpredictable (the agent, the sandbox, the verification). Atina takes well-scoped pieces that each have a clear finish line, good documentation online, and can be built and tested alone. Satyabrata reviews every one of Atina's pull requests within a day, and a fake version of each side means neither of us waits on the other.

Satyabrata: the risky core

- **Day-one test:** by hand, in a chat window, check that a Nemotron model can fix the seeded bug. What we learn decides how much effort the agent loop needs.
- Nebius client for the Nemotron models: chat calls, retries, timeouts, token counting.
- The agent loop: diagnose, pick files to read, propose a patch, read test output, revise, stop after a fixed number of tries.
- Prompts and context management, so long runs do not exhaust the context window.
- E2B sandbox runner: create a sandbox, clone the repo, install dependencies, run commands, apply a diff, tear down.
- Verification harness: run the new fail-to-pass test and the full existing suite, and report a clear pass or fail.
- Patch safety checks: reject diffs that touch test files or files outside the repo.

- Hosted demo URL, secrets and keys.
- Integration: plugs Atina's pieces into the agent, owns the end-to-end run, reviews all code.

Atina: well-scoped pieces

Each piece has a "done when" line so it is clear when it is finished.

Piece	What it is	Done when
1. Demo target app	A small Python service with a pytest suite and two or three seeded bugs, each failing the same way every time	`pytest` shows exactly the planned failures, and a one-line fix makes each one pass
2. Alert simulator	A script that sends a realistic alert with a stack trace to the webhook	One command triggers a run
3. Webhook service	A FastAPI endpoint that checks a shared secret and turns a Python traceback into an incident (file, line, function per frame)	Posting the sample alert returns the parsed incident
4. Tavily module	A function that searches for an error message or library problem and returns titles, links and snippets, plus a Sources list formatter	A sample error returns three cited results
5. Post-mortem template	A function that fills a markdown template (root cause, fix, how it was verified, sources) from a run result	A sample run result produces a clean pull request description
6. GitHub pull request step	A script that creates a branch, commits a diff and opens a pull request	A pull request opens on the demo repo from one command
7. Live progress view	A terminal display (a web page only if time allows) showing each step of a run	It plays back a recorded sample run

Piece	What it is	Done when
8. README, video and feedback log	README draft, 3-minute video script and recording, and running notes on Nebius and NVIDIA tools	A fresh machine can run the demo from the README; the feedback log grows weekly

As a beginner, Atina's honest notes on setup and documentation are exactly the feedback the Most Valuable Feedback prize asks for, so keep the log from day one.

How we work together

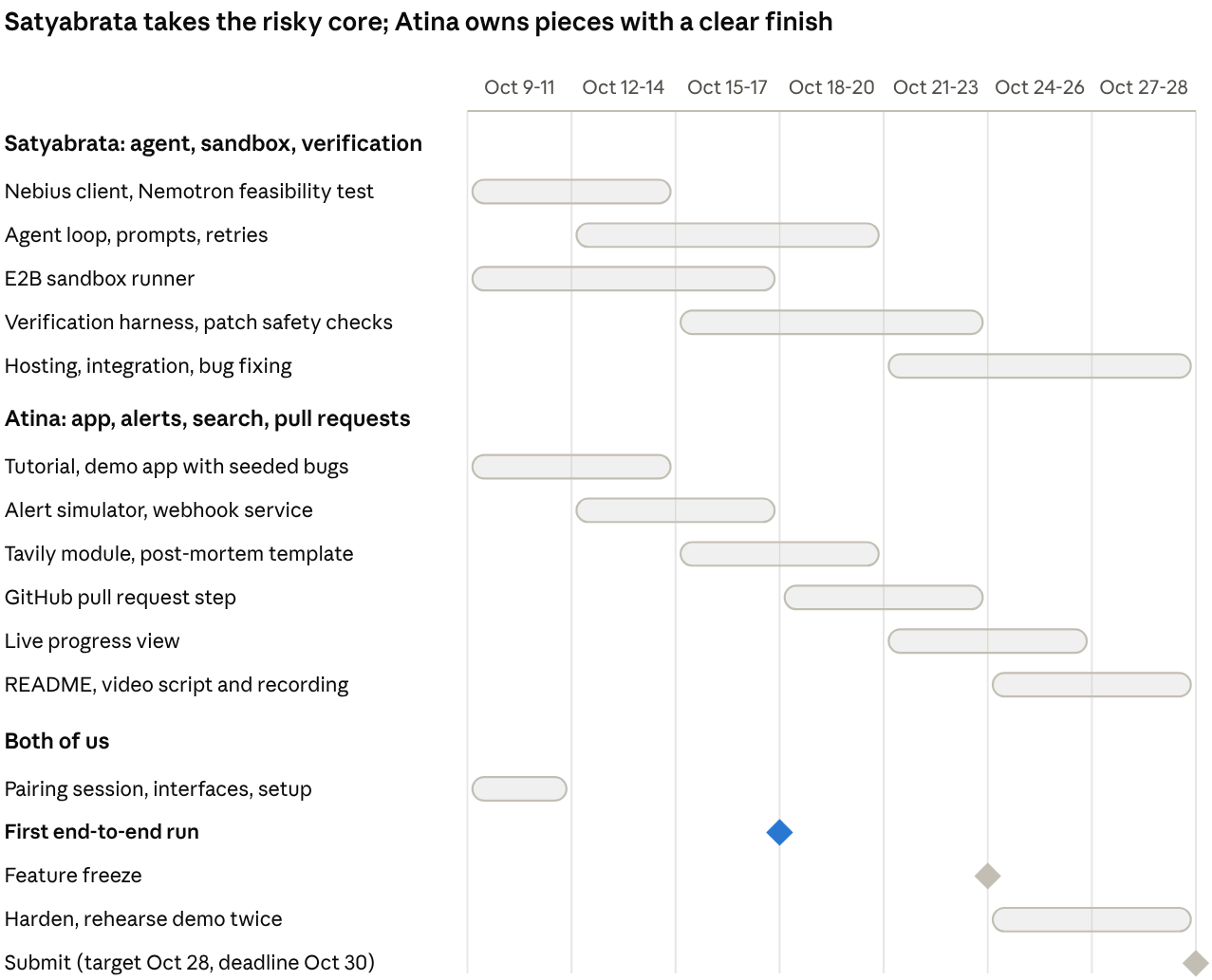
- **Oct 9:** a two-hour pairing session to create the repo, run a hello world in each tool, and agree the three interfaces below.
- Atina starts with a short FastAPI and pytest tutorial (two to three hours) before piece 1.
- Pull requests stay small. Satyabrata reviews Atina's within a day, and Atina reads his too; reading working code is the fastest way to learn.
- If either of us is stuck for more than two hours, ask the other.
- Satyabrata gives Atina a canned run result early, and Atina gives Satyabrata a canned incident and search result, so both can test alone.
- If Atina is behind on Oct 17, Satyabrata takes piece 6 and the live view stays terminal-only.
- Integration testing together from Oct 14, and the demo rehearsed twice by both of us before recording.

Interfaces to agree on first

Interface	Produced by	Used by	Shape
Incident	Atina (webhook)	Satyabrata (agent)	JSON: error type, message, stack frames (file, line, function), repo URL, commit SHA, timestamp
Search result	Atina (Tavily module)	Satyabrata (agent)	List of title, URL and snippet

Interface	Produced by	Used by	Shape
Run result	Satyabrata (agent)	Atina (post-mortem, pull request, live view)	JSON: status (fixed or failed), final diff, iteration count, test results, sources used, list of timestamped step events

Timeline



build plan Oct 9 to Oct 28 · two lanes plus shared milestones

The deadline is Oct 30, 10:00am PDT (1:00pm ET), so we plan to finish on Oct 28 and keep two days of buffer. We work in parallel until the first full run from alert to pull request on Oct 17, even if rough. New features stop on Oct 23; the last week is for reliability, the hosted demo, the video and the feedback write-up.

- [ ] Do not work down to the deadline: the plan targets Oct 28, and the last two days are only for emergencies.
- [ ] Daily 15-minute sync: what is done, what is blocked, what changes in the interfaces.
- [ ] By Oct 14: swap stubs for real components and run a smoke test together.

Demo plan and submission

The demo is a two to three minute story: something breaks, Aftermath fixes it, and the proof is on screen.

1. **Hook (15 sec):** show the demo app returning an error to a user.
2. **Trigger (15 sec):** the alert fires; the live view shows the incident arriving.
3. **Work (60 sec):** the agent reads code, a sandbox starts, the bug reproduces with a failing test, the agent patches and retries once visibly.
4. **Proof (30 sec):** the new test goes green and the full suite stays green.
5. **Result (30 sec):** open the real pull request and its post-mortem.
6. **Close (15 sec):** one line on why verification matters and where this goes next.

Demo safeguards: pre-record a backup video of a successful run; keep the seeded bugs deterministic; cache a known-good run so the demo still works if the network or API is slow.

Submission checklist:

- [ ] Public repo with an open source license (MIT or Apache 2.0) visible at the top, and a README with setup steps, architecture diagram and how to run the demo; highlight Nemotron, Token Factory and Tavily
- [ ] Public YouTube video of 3 minutes or less showing the project working and how we used Token Factory and Nemotron
- [ ] Project description: what we built, why, and how it works, naming the Nebius and NVIDIA pieces used
- [ ] A hosted demo URL (required for this track) and a real generated PR as proof
- [ ] No secrets in the repo history (scan before pushing)
- [ ] Submit by Oct 28; the deadline is Oct 30, 10:00am PDT (1:00pm ET)
- [ ] Written feedback on Nebius Token Factory, AI Cloud and the NVIDIA tools we used (required; also enters us for Most Valuable Feedback)
- [ ] Choose the Coding and Agentic Engineering track on the submission form

Risks and scope cuts

The biggest risk is integration friction between the agent and the sandbox, so we connect them with stubs in the first two days and never leave them untested together for long.

Risk	Mitigation	Owner
Agent loops on failing tests and exhausts context	Hard cap on iterations (for example 5); trim logs to the failing section; summarize prior attempts instead of resending them	Satyabrata
Model on Nebius is weaker than expected at producing valid diffs	Compare Nemotron 3 Ultra, Super and Nano in the first two days; ask for whole-function rewrites if diffs fail to apply	Satyabrata
Sandbox setup is slow or flaky	Pre-build a snapshot or image with dependencies installed; Daytona as fallback; local Docker as last resort	Satyabrata
Stack-trace parsing breaks on odd formats	Support one format well (Python tracebacks) and say so	Atina
Agent cheats by editing or deleting tests	Reject any diff that touches test files; run the original test files from a clean copy	Satyabrata
Atina's pieces fall behind	A "done when" line per piece, reviews within a day, canned data so nobody waits; Satyabrata takes the pull request step if behind on Oct 17	Both
Live demo fails on stage	Pre-recorded backup, cached known-good run, deterministic seeded bugs	Both
Scope creep	Freeze features on Oct 23; polish only after that	Both

Scope cuts, in order, if time runs short: drop the automated review pass, then the web progress view (use terminal output), then extra seeded bugs (keep one), then Tavily lookups. Never cut verification or the real PR; those are the product.

Check the source claims: the evaluation document we started from makes confident statements (specific model names, benchmark numbers, "near-perfect score"). Treat those as hypotheses. Its model name (Nemotron-4 340B) differs from the hackathon page, which lists Nemotron 3 Ultra, Super and Nano; follow the hackathon page. Verify E2B or Daytona capabilities ourselves, and do not repeat unverified figures in our submission.